

(بخش ب)

خلاصه عملکرد:

1. کلاس InterruptDemo شامل دو بخش اصلی است:
 - تابع interrupt_handler که نقش روال سرویس وقفه (ISR) را دارد.
 - تابع main_program که حلقه‌ی اصلی برنامه را اجرا می‌کند.
2. در ابتدای اجرای برنامه، سیگنال SIGINT (که هنگام فشردن Ctrl+C ایجاد می‌شود) به تابع interrupt_handler متصل می‌شود.

این یعنی هر زمان کاربر Ctrl+C را بزند، اجرای برنامه موقتاً متوقف شده و کنترل به این تابع منتقل می‌شود.
3. در حلقه‌ی اصلی، برنامه به‌صورت مداوم اعداد پشت سر هم را نمایش می‌دهد تا زمانی که کاربر وقفه ایجاد کند.
4. وقتی Ctrl+C فشرده می‌شود:
 - تابع interrupt_handler اجرا می‌شود.
 - اطلاعاتی مانند شماره وقفه، زمان فعلی و پیغام‌های وضعیت چاپ می‌شود.
 - برنامه با دستور time.sleep(5) پنج ثانیه شبیه‌سازی می‌کند که گویی مشغول پردازش در ISR است.
 - پس از اتمام، کنترل به برنامه‌ی اصلی برمی‌گردد و شمارش ادامه می‌یابد.
5. در نهایت اگر کاربر با Ctrl+C یا بستن برنامه آن را خاتمه دهد، پیامی با تعداد کل وقفه‌های ثبت‌شده نمایش داده می‌شود.

نتیجه:

این کد به‌صورت ساده نشان می‌دهد که چگونه می‌توان در پایتون، با استفاده از سیگنال‌ها، یک سیستم وقفه نرم‌افزاری را پیاده‌سازی کرد تا هنگام رخداد وقفه، کنترل برنامه به تابع خاصی منتقل شده و پس از پردازش، به اجرای عادی بازگردد.

(بخش ج)

این نسخه از برنامه یک شبیه‌سازی پیشرفته از مدیریت وقفه‌های سیگنالی (Signal Handling) در پایتون است که دو نوع سیگنال را همزمان مدیریت می‌کند:

- SIGINT (Ctrl + C)
- SIGTSTP (Ctrl + Z)

ساختار کلی برنامه:

برنامه از یک کلاس به نام InterruptDemo تشکیل شده که دارای چند بخش اصلی است:

1.

سازندهی کلاس (`_init__`)

در این بخش، متغیرهای زیر مقداردهی می‌شوند:

- شمارندهی وقفه‌ها (`sigint_count` و `sigstp_count`)
- وضعیت اجرای برنامه (`running`)
- پرچم‌های کنترلی برای هماهنگی بین وقفه‌ها (`sigstp_received`, `waiting_for_sigstp`)
- قفل `threading` برای هماهنگی احتمالی بین نخ‌ها

2.

تابع (`sigint_handler`) مدیریت `Ctrl + C`

زمانی که کاربر کلید `Ctrl + C` را فشار می‌دهد، این تابع اجرا می‌شود و مراحل زیر را طی می‌کند:

1. شمارندهی `SIGINT` افزایش می‌یابد.
2. اطلاعاتی مانند شماره وقفه و زمان فعلی چاپ می‌شود.
3. یک بخش پردازشی کوتاه (“فاز اول”) شبیه‌سازی می‌شود.
4. برنامه منتظر می‌ماند تا کاربر `Ctrl + Z` بزند.
5. وقتی `SIGTSTP` دریافت شد، وارد “فاز دوم پردازش” می‌شود و کار وقفه‌ی `SIGINT` تکمیل می‌شود.
6. سپس کنترل دوباره به برنامه‌ی اصلی برمی‌گردد.

نکته: اگر در زمان انتظار دوباره `Ctrl+C` زده شود، پیام “Already waiting for SIGTSTP” چاپ می‌شود و وقفه‌ی جدید نادیده گرفته می‌شود.

3.

تابع (`sigstp_handler`) مدیریت `Ctrl + Z`

این تابع دو حالت دارد:

- اگر برنامه در حالت عادی باشد، `Ctrl + Z` باعث اجرای یک وقفه‌ی مستقل `SIGTSTP` می‌شود که کار کوتاهی (۲ ثانیه) انجام داده و سپس به برنامه‌ی اصلی برمی‌گردد.

- اگر در هنگام اجرای SIGINT منتظر Ctrl+Z باشیم، زدن Ctrl+Z به جای اجرای مستقل، فقط علامت می‌دهد که “سیگنال دریافت شده” تا فاز دوم SIGINT آغاز شود.

4.

تابع main_program

- در این تابع، هندلرهای SIGINT و SIGTSTP به توابع مربوطه متصل می‌شوند.
- سپس یک حلقه بی‌نهایت شروع می‌شود که اعداد متوالی را روی صفحه چاپ می‌کند (شبیه کار معمول برنامه).
- در هر لحظه کاربر می‌تواند با Ctrl+C یا Ctrl+Z وقفه‌ها را فعال کند.
- با فشردن Ctrl+\ (یا قطع اجرای برنامه)، حلقه متوقف شده و آمار نهایی وقفه‌ها چاپ می‌شود.

نتیجه‌ی عملکرد:

فشاردن Ctrl + C

اجرای فاز اول SIGINT

برنامه منتظر Ctrl + Z می‌ماند

پس از فشردن Ctrl + Z، فاز دوم SIGINT اجرا شده و برنامه به حالت عادی برمی‌گردد.

فشاردن Ctrl + Z به‌تنهایی

اجرای وقفه‌ی مستقل SIGTSTP (بدون وابستگی به SIGINT)

فشاردن Ctrl + \

پایان اجرای برنامه و چاپ آمار وقفه‌ها